

Evento Cultural

Documentación Técnica del Proyecto

Plataforma chilena de eventos culturales

Integrante	Rol principal
Roberto Rizzo	Líder de proyecto / Frontend / UX
José Miguel Moncada	Base de datos / QA
David Herrera	Backend e integraciones

Roles según el design brief aprobado en Fase 1 (*design_brief_capstone.docx*).

Stack del sistema: Python (scraper con conectores, NVIDIA NIM y checkpoints SQ-Lite), Supabase/PostgreSQL con Row Level Security, Angular 22 + Ionic 9 (web y móvil), GitHub Actions para automatización.

Versión 1.0

24 de septiembre de 2026

Documento elaborado exclusivamente con evidencia verificable del repositorio.
Las omisiones se declaran como **pendientes del equipo** y están trazadas en el *Registro de Cambios de Documentación*.

Índice

1. Introducción y alcance	3
1.1. Estructura del sistema	3
2. Arquitectura general del sistema	3
2.1. Visión general	3
2.2. Comunicación entre servicios	3
2.3. Estado de la integración frontend–base de datos	3
3. Arquitectura del scraper	4
3.1. Modelo general	4
3.2. Conectores y fuentes	5
3.3. Enriquecimiento selectivo con NVIDIA NIM	5
3.4. Pipeline reanudable y concurrencia	6
3.5. Patrones de diseño aplicados	6
3.6. Pendientes de la arquitectura del scraper	6
4. Arquitectura del frontend	6
4.1. Stack real	6
4.2. Estructura y navegación	6
4.3. Filtros y funcionalidad	7
4.4. Estrategia de despliegue	7
4.5. Pendientes del frontend	7
5. Modelo de datos (Supabase / PostgreSQL)	8
5.1. Fuente del modelo	8
5.2. Tablas y finalidad	8
5.3. Seguridad (Row Level Security) y ciclo de carga	9
5.4. Limitaciones declaradas	9
6. Diagramas UML	9
6.1. Casos de uso	10
6.2. Secuencia de la funcionalidad principal	10
6.3. Componentes y clases	10
6.4. Pendientes de UML	11
7. Requisitos no funcionales	11
7.1. Requisitos definidos en Fase 1 (MoSCoW)	11
7.2. Estado real por requisito	11
7.3. Requisitos declarados y no implementados	12
7.4. Limitaciones técnicas del desarrollo	12
8. Pruebas	13
8.1. Pruebas existentes del scraper	13
8.2. Pruebas existentes del frontend	13
8.3. Pruebas pendientes	14
8.4. Registro de resultados	14
9. Docker: estado actual y plan	14
9.1. Estado real verificado	14
9.2. Plan de implementación (alternativa documentada, no adoptada)	14

10.Innovación	15
10.1. ¿Qué problema resuelve?	15
10.2. ¿Qué hace diferente a la solución?	16
10.3. ¿Qué valor agrega?	16
10.4. Modelo de negocio (propuesto, no implementado)	16
11.Trazabilidad documental	17

1. Introducción y alcance

Este documento consolida la documentación técnica formal del proyecto **Evento Cultural**, una plataforma chilena para compilar y difundir eventos culturales, comerciales y municipales. Fue elaborado a partir de la evidencia disponible en el propio repositorio: el código y la documentación interna del scraper (**Scraper de eventos culturales.rar**), el código y la documentación del frontend (**evento-cultural.rar**), el modelo de base de datos (DBML, migraciones documentadas y diccionario de tablas), los requerimientos priorizados en Fase 1 (matriz MoSCoW) y el diagrama de arquitectura de Fase 2.

Convención de honestidad documental: este documento distingue explícitamente entre lo *implementado* (respaldado por código), lo *parcial* (respaldado solo en parte) y lo *pendiente* (información humana o trabajo futuro). Nada se afirma como terminado sin respaldo; los puntos abiertos se marcan con **Pendiente de completar por el equipo**.

1.1. Estructura del sistema

El sistema se compone de tres bloques independientes que se comunican a través de la base de datos: el **scraper** (recolección y enriquecimiento de datos), la **base de datos** (catálogo canónico en Supabase/PostgreSQL) y el **frontend** (aplicación web y móvil), con **GitHub Actions** como automatización de la ejecución del scraper (ver Figura 1 en la Sección 2).

2. Arquitectura general del sistema

2.1. Visión general

El sistema sigue una arquitectura de tres bloques desacoplados que se comunican a través de la base de datos, más un actor de automatización:

- **Scraper (Python):** recolecta eventos desde fuentes públicas chilenas, los normaliza y los enriquece selectivamente con NVIDIA NIM.
- **Base de datos (Supabase/PostgreSQL):** catálogo canónico con Row Level Security, bitácora de ejecuciones y evidencia de extracción.
- **Frontend (Angular 22 + Ionic 9):** aplicación web y móvil para descubrir, filtrar y publicar eventos.
- **GitHub Actions:** automatiza la ejecución del scraper, ejecuta las pruebas y sincroniza la base de datos.

2.2. Comunicación entre servicios

Cada fuente se procesa de manera independiente: si una falla, las demás continúan. Solo se publican eventos vigentes; la conciliación retira registros antiguos únicamente cuando la extracción fue completa y segura (diagrama de arquitectura de Fase 2).

2.3. Estado de la integración frontend–base de datos

El frontend entregado funciona con **datos de ejemplo en memoria** (`src/app/data/mock-eventos.ts`) según su propio README; los formularios de inicio de sesión y publicación simulan el flujo. La documentación del scraper y del frontend define el punto de integración futuro (**EventosService**). **Pendiente de completar por el equipo:** cronograma y responsables de la conexión real a Supabase y de la autenticación real.

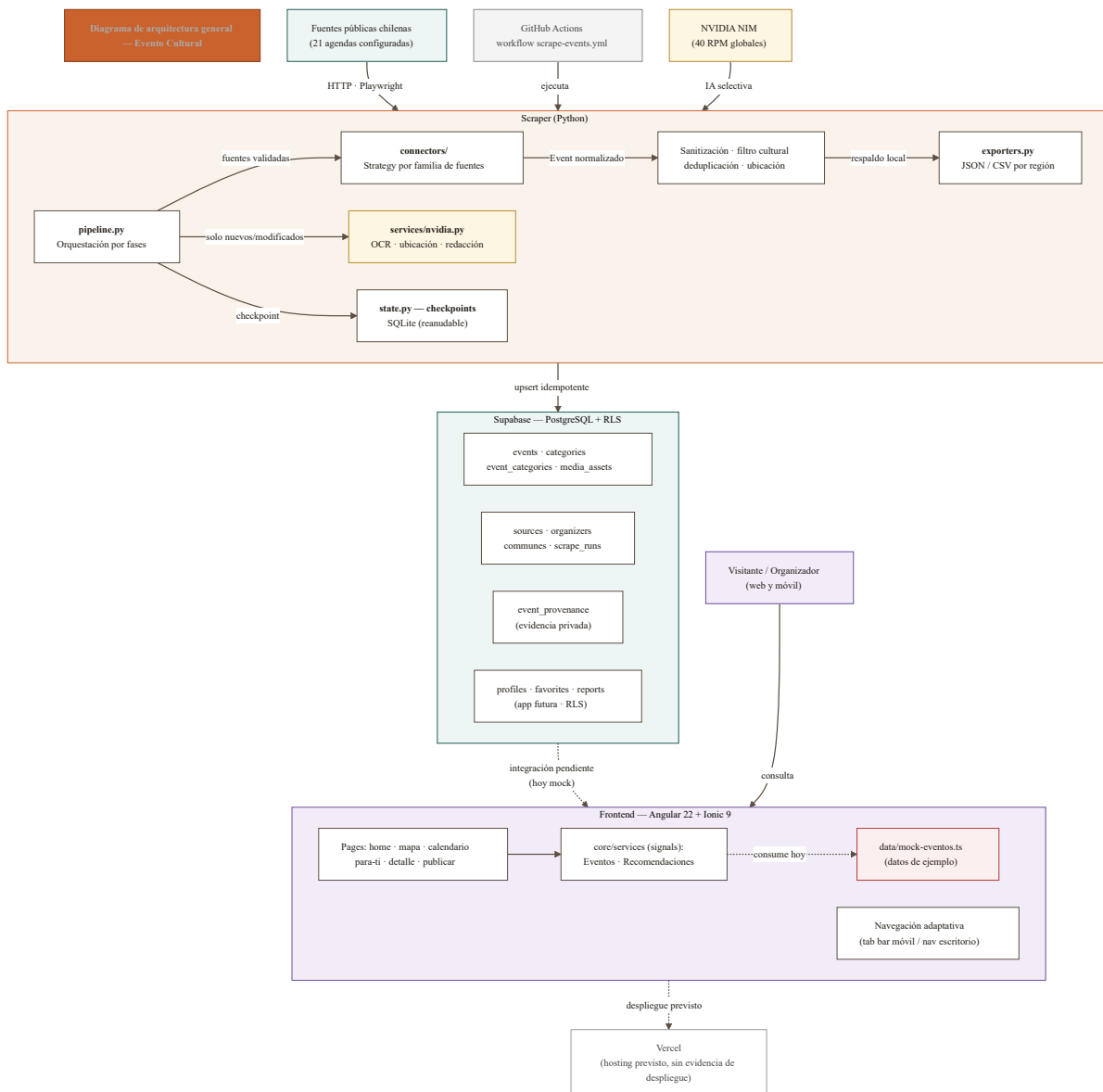


Figura 1: Arquitectura general del sistema (fuente editable: `Diagramas/01_arquitectura_general.mmd`, Mermaid).

3. Arquitectura del scraper

3.1. Modelo general

Pipeline por fases con conectores intercambiables. Los ejecutables de la raíz (`scrape_events.py`, `add_source.py`, `validate_sources.py`) reciben la orden; el trabajo reside en el paquete `eventos/`. El flujo principal, según `docs/ARCHITECTURE.md`, es:

1. `SourceRepository` carga y valida los archivos de `sources/`.
2. El pipeline agrupa fuentes por dominio y las procesa con un pool acotado.
3. `ConnectorFactory` selecciona una `Strategy` según el campo `connector`; cada conector devuelve objetos `Event` con el mismo esquema.
4. `EventSanitizer` limpia los campos, limita descripciones y descarta vencidos.

Origen → Destino	Mecanismo	Detalle
GitHub Actions → Scraper	Ejecución de <code>scrape_events.py</code>	Fases separables (<code>--phase extract / --phase process</code>); estado restaurado por artefactos
Scraper → Fuentes	HTTP directo o Playwright	Caché condicional (ETag/Last-Modified), límites por dominio, respeto de <code>robots.txt</code> y <code>Retry-After</code>
Scraper → NVIDIA NIM	API de inferencia	Limitador global de 40 RPM con reintentos
Scraper → SQLite	Checkpoints por fuente y ficha	Reanudación de OCR/IA pendiente en futuras ejecuciones
Scraper → Supabase	API REST (upserts idempotentes)	Carga por lotes (<code>SUPABASE_BATCH_SIZE</code>); borrado de vencidos y conciliación configurables
Frontend → Supabase	Cliente/API REST	Pendiente: hoy consume datos de ejemplo en memoria

Cuadro 1: Comunicación entre servicios del sistema.

5. Deduplicación por fuente y global, fijando el identificador estable.
6. `LocationNormalizer` aplica valores por defecto, reglas y, solo si falta información, IA.
7. Filtro regional y ubicación canónica.
8. `DatasetExporter` escribe el respaldo local (JSON/CSV por región).
9. Si Supabase está habilitado, `SupabaseExporter` sincroniza con upserts idempotentes y elimina vencidos remotos.

3.2. Conectores y fuentes

Conector	Uso
<code>wordpress_rest</code>	APIs WordPress REST con <code>field_map</code> declarativo
<code>json_api</code>	APIs JSON genéricas con paginación
<code>json_ld</code>	Sitios con datos estructurados Schema.org
<code>html_cards</code>	Carteleras HTML estáticas con selectores CSS declarativos
<code>eventon, tribe</code>	Plugins de eventos (EventON, The Events Calendar)
<code>ticketplus, maipu, fisa</code>	Conectores especializados
<code>generic</code>	Busca JSON-LD primero; usa NVIDIA NIM solo si faltan datos estructurados

Cuadro 2: Conectores disponibles en el paquete `eventos/connectors/`.

Las fuentes se declaran en `sources/*.json`: 20 permanentes más 1 estacional de septiembre (tabla completa en el README del scraper), con plantillas por familia en `sources/templates/`.

3.3. Enriquecimiento selectivo con NVIDIA NIM

Solo los eventos nuevos o modificados pasan por OCR, normalización de ubicación y redacción editorial; los eventos sin cambios se reutilizan por hash de entrada. Las tareas pendientes quedan en la cola persistente del checkpoint SQLite y se recuperan en ejecuciones futuras. El limitador global `NVIDIA_RPM_LIMIT=40` espacia las peticiones aunque varios conectores terminen simultáneamente.

3.4. Pipeline reanudable y concurrencia

Cada fuente y resultado de IA se guarda en `state/pipeline.sqlite3` (ver `docs/RESUMABLE_PIPELINE.md`). El workflow de GitHub Actions restaura y conserva este estado mediante artefactos. Con `SOURCE_WORKERS=4` se crean trabajadores internos; las fuentes de un mismo dominio se procesan en serie (*bulkhead* por dominio) y la salida es determinista. La caché HTTP condicional persistente (ETag/Last-Modified) evita descargas innecesarias.

3.5. Patrones de diseño aplicados

Strategy (conectores), Registry/Factory, Repository, inyección de dependencias para pruebas, Adapter (`SupabaseExporter`), Chain of Responsibility (ubicación), Rate Limiter global (NVIDIA), Bulkhead por dominio, Cache-Aside condicional y post-processing pipeline obligatorio de saneamiento y vigencia (un conector nuevo no puede saltárselo).

3.6. Pendientes de la arquitectura del scraper

- La periodicidad del workflow quedó resuelta: el bloque `schedule` está activo desde el 24-09-2026 con cron diario a las 00:37 (America/Santiago), verificado en GitHub Actions (corridas #9–#11 del 25 al 27-09-2026). Queda como mejora pendiente congelar el reloj en las pruebas de vigencia para que las corridas programadas no fallen por fechas simuladas vencidas (ver sección de pruebas). **Pendiente de completar por el equipo: primera ejecución automática completa con la suite en verde.**
- La carpeta `supabase/` con las migraciones SQL no está incluida en la copia entregada (la documentación la referencia). **Pendiente de completar por el equipo: confirmar su presencia en el repositorio oficial.**

4. Arquitectura del frontend

4.1. Stack real

Componente	Versión
Angular	22.0.1 (componentes standalone, sin NgModules)
Ionic Framework	^9.0.0
Capacitor	8.x (app, haptics, keyboard, status-bar)
TypeScript	~6.0.0
Pruebas	Vitest ^4.0.8 (con jsdom)
Lint	ESLint ^9.16.0 + angular-eslint 22.0.0

Cuadro 3: Stack del frontend según `package.json`.

4.2. Estructura y navegación

La aplicación organiza el código en `core/` (modelos `Evento` y `Recomendacion`; servicios con signals: `EventosService`, `RecomendacionService`, `ToastService` y `ViewPreferencesService`), `features/home/` (descubrimiento y layouts de inicio), `layout/` (`MainLayoutComponent`) y `pages/`. Las rutas registradas (`app.routes.ts`) son: `/home`, `/para-ti`, `/calendario`, `/mapa`, `/evento/:id`, `/publicar`, `/perfil` y `/login`, con redirecciones de compatibilidad (`/buscar` y `/favoritos`).

La navegación es adaptativa sin JavaScript adicional: tab bar inferior en móvil y barra superior en escritorio, ambas dentro de `MainLayoutComponent` mediante las clases `responsive` de Ionic. La

identidad visual (paleta terracota/verde/ámbar y tipografías Fraunces/Inter) está centralizada en `src/theme/variables.scss`.

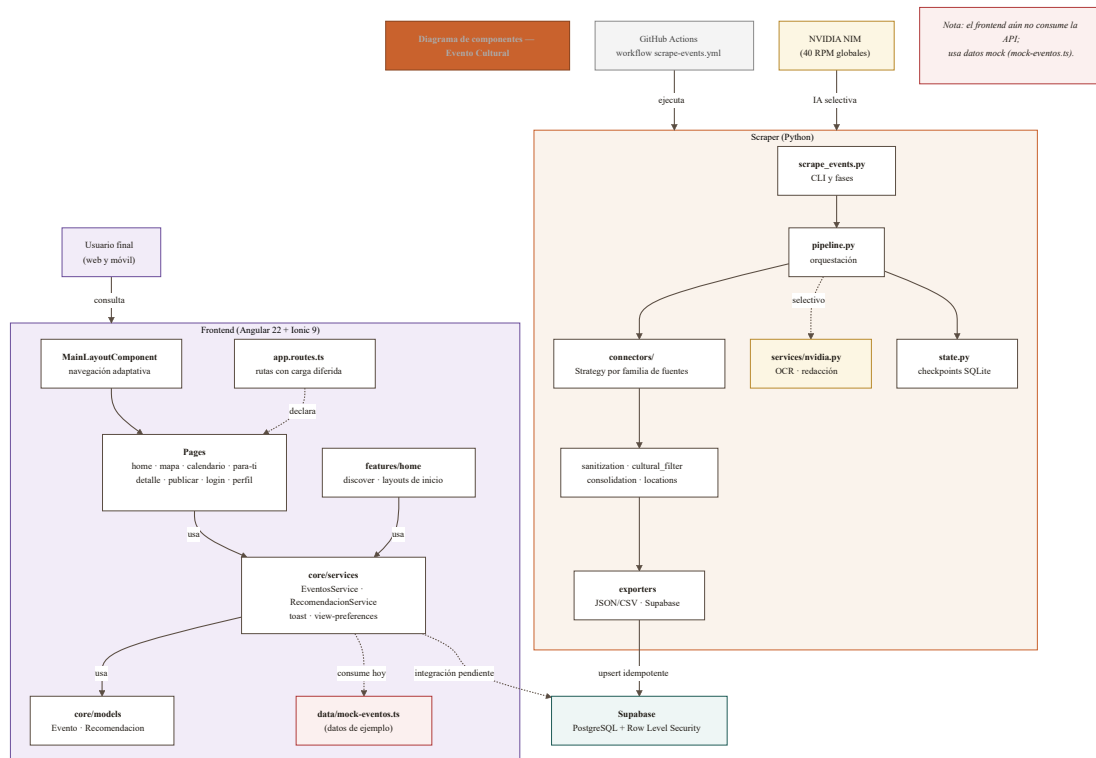


Figura 2: Diagrama de componentes del sistema (fuente editable: `Diagramas/02_componentes.mmd`, Mermaid).

4.3. Filtros y funcionalidad

`EventosService` implementa orden por fecha, nombre, categoría, cercanía y actualización, y filtros combinables: texto, categorías múltiples, región, comuna, rango de fechas, gratuito, modalidad (presencial/virtual/híbrido), público objetivo y accesibilidad. La vista “para ti” (`RecomendacionService`) sustituyó a favoritos como experiencia de recomendación.

4.4. Estrategia de despliegue

Decisión del equipo (comunicada el 24-09-2026), consistente con el stack definido en Fase 1:

- **Web:** despliegue en **Vercel** (ya figuraba como hosting previsto en el design brief de Fase 1). A la fecha no existe evidencia de un despliegue ejecutado.
- **Android:** la aplicación móvil se compila con **Capacitor 8** (dependencias presentes en `package.json`) y **Android Studio**.
- **Scraper:** automatización con **GitHub Actions** (workflow `scrape-events.yml`).
- **No se utilizará Docker** como mecanismo de despliegue (ver Sección 9).

4.5. Pendientes del frontend

Pendiente de completar por el equipo: conexión real a Supabase (sustituir los datos mock por el cliente de la API), activación de la autenticación real y persistencia de favoritos del usuario.

5. Modelo de datos (Supabase / PostgreSQL)

5.1. Fuente del modelo

El modelo se documenta a partir de docs/DATABASE.md, docs/SUPABASE.md, el DBML editable (docs/evento_cultural_modelo.dbml), el diccionario de tablas (DOCX, versión documentada al 21-09-2026) y el diagrama ER de Fase 1 (DB_evento_cultural.pdf). **Pendiente de completar por el equipo:** la carpeta supabase/migrations/ con los archivos SQL no está incluida en la copia entregada; confirmar su presencia en el repositorio oficial.

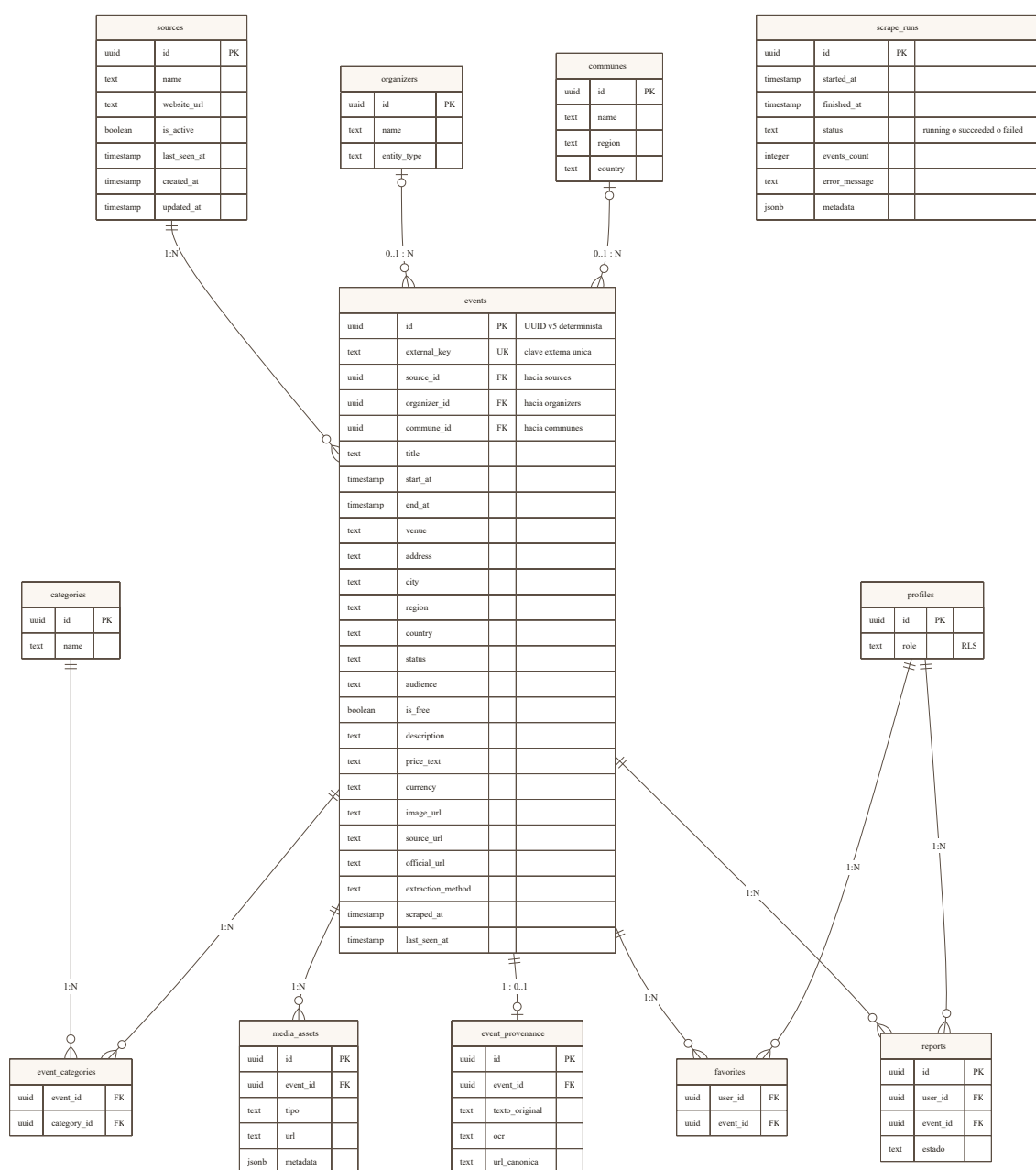


Figura 3: Modelo entidad-relación (fuente editable: Diagramas/06_modelo_er.mmd, Mermaid).

5.2. Tablas y finalidad

Los identificadores de eventos, organizadores, comunas y categorías son **UUID v5 deterministas** derivados de claves externas: ejecutar dos veces el mismo scraping actualiza las filas (up-

Grupo	Tabla	Finalidad
Ingesta y catálogo	<code>sources</code>	Agenda, municipalidad, ticketera o sitio de origen
	<code>organizers</code>	Institución responsable del evento
	<code>communes</code>	Ubicación administrativa normalizada
	<code>categories</code>	Etiquetas temáticas, de formato o audiencia
	<code>events</code>	Registro canónico del evento consolidado (37 columnas)
	<code>event_categories</code>	Relación N:M entre eventos y categorías
	<code>media_assets</code>	Imágenes y futuros multimedia
	<code>scrape_runs</code>	Bitácora de ejecuciones locales y de GitHub Actions
	<code>event_provenance</code>	Evidencia privada: texto original, OCR y URL canónica
Aplicación futura	<code>profiles,</code> <code>reports</code>	<code>favorites,</code> Base prevista de la aplicación; no intervienen en el scraper

Cuadro 4: Tablas del modelo de datos según el diccionario y el DBML del proyecto.

sert) sin crear duplicados. El campo `categories` se normaliza en `categories` y `event_categories`, conservando la relación muchos-a-muchos.

5.3. Seguridad (Row Level Security) y ciclo de carga

La migración de RLS habilita Row Level Security en todas las tablas; `scrape_runs` no tiene lectura pública; el usuario no puede cambiar su propio rol ni el estado de sus reportes desde el frontend; la vista `public_event_catalog` hereda las reglas RLS y no expone eventos suprimidos ni organizadores bloqueados.

En el ciclo de carga, la sincronización identifica coincidencias por URL o clave compuesta, carga por lotes (`SUPABASE_BATCH_SIZE`) y registra cada ejecución en `scrape_runs` como `running`, `succeeded` o `failed`. La repetición idempotente es el mecanismo de recuperación ante cargas parciales.

5.4. Limitaciones declaradas

Sin marca explícita de eventos de día completo; `price_text` no permite comparación numérica; categorías sin jerarquía almacenada; registros de Chile Cultura que pueden ser directorios y no eventos fechados; etiqueta inválida conocida (`Calendar_Month Inscribirse`); borrado físico de vencidos solo con `SUPABASE_DELETE_EXPIRED=true`.

6. Diagramas UML

Los cuatro diagramas UML exigidos por el anexo metodológico se escriben como código fuente Mermaid (`.mmd`) y se renderizan a PDF con `mermaid-cli`, en la carpeta `Diagramas/` de esta misma sección. Los casos de uso se derivan de la matriz MoSCoW de Fase 1 y de las páginas realmente implementadas en el frontend. Todos los diagramas (01–06) comparten un estándar visual único (paleta institucional, tipografía Arial, flechas reforzadas), declarado dentro de cada archivo `.mmd`, y fueron regenerados el 27-09-2026 al migrar de Draw.io a Mermaid para automatizar el trazado de líneas y facilitar el mantenimiento (detalle en el `REGISTRO_CAMBIOS_DOCUMENTACION.md`, §16.3).

6.1. Casos de uso

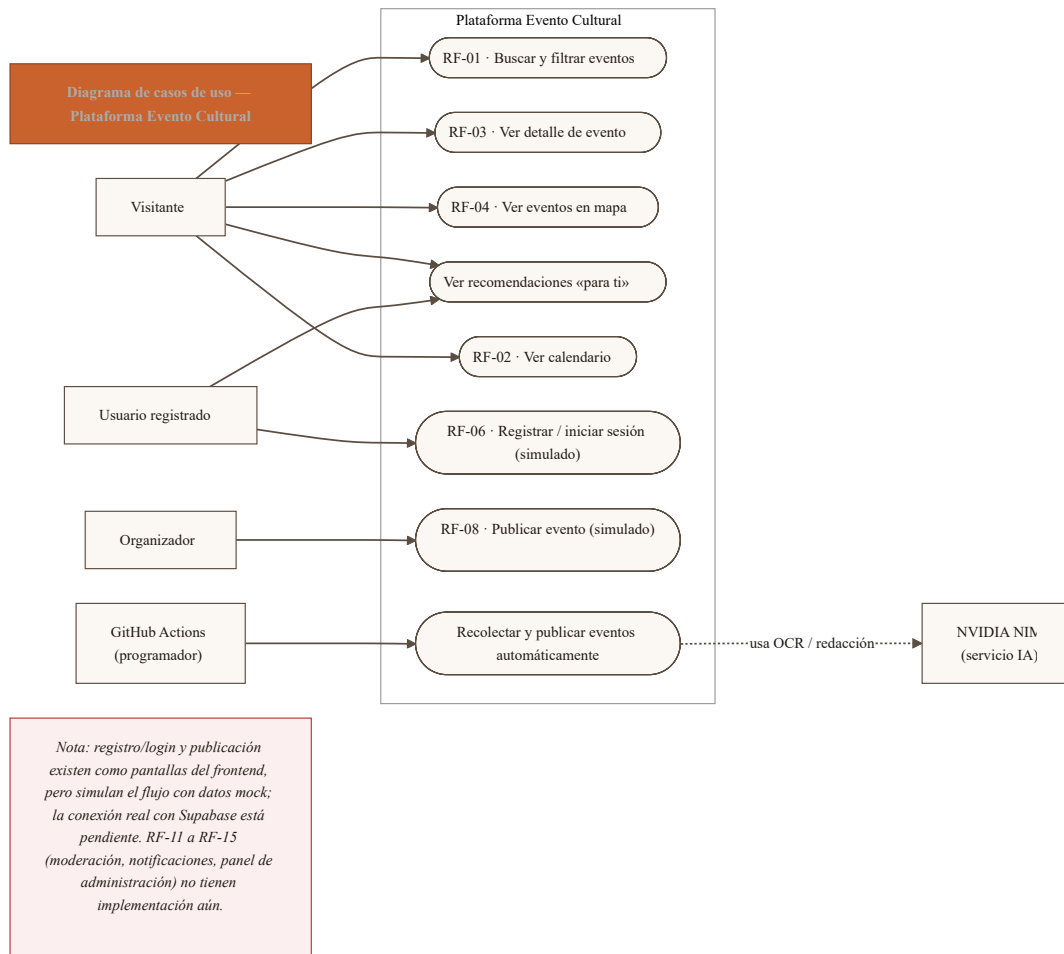


Figura 4: Casos de uso de la plataforma (fuente editable: `Diagramas/03_casos_de_uso.mmd`, Mermaid).

El visitante busca, filtra, ve el detalle, consulta el mapa y el calendario; el usuario registrado inicia sesión y recibe recomendaciones; el organizador publica eventos (simulado); GitHub Actions activa la recolección automática, que usa NVIDIA NIM como servicio externo de OCR y redacción.

6.2. Secuencia de la funcionalidad principal

El flujo principal del sistema es la obtención y publicación de eventos: GitHub Actions ejecuta el pipeline; este restaura el checkpoint, consulta las fuentes con caché condicional, sana y deduplica, decide mediante hash qué eventos pasan por IA (con límite de 40 RPM), reutiliza resultados previos y sincroniza Supabase con upserts idempotentes, registrando la corrida en `scrape_runs`.

6.3. Componentes y clases

El diagrama de componentes (Figura 2) muestra los módulos del frontend y del scraper y su comunicación con la base de datos. El diagrama de clases (conceptual) refleja los roles reales del núcleo del scraper documentados en `docs/ARCHITECTURE.md`: `EventPipeline`, `ConnectorFactory`, `Connector` (Strategy), `Event`, `EventSanitizer`, `LocationNormalizer`, `NvidiaService`, `SupabaseExporter`, `DatasetExporter` y `PipelineState`.

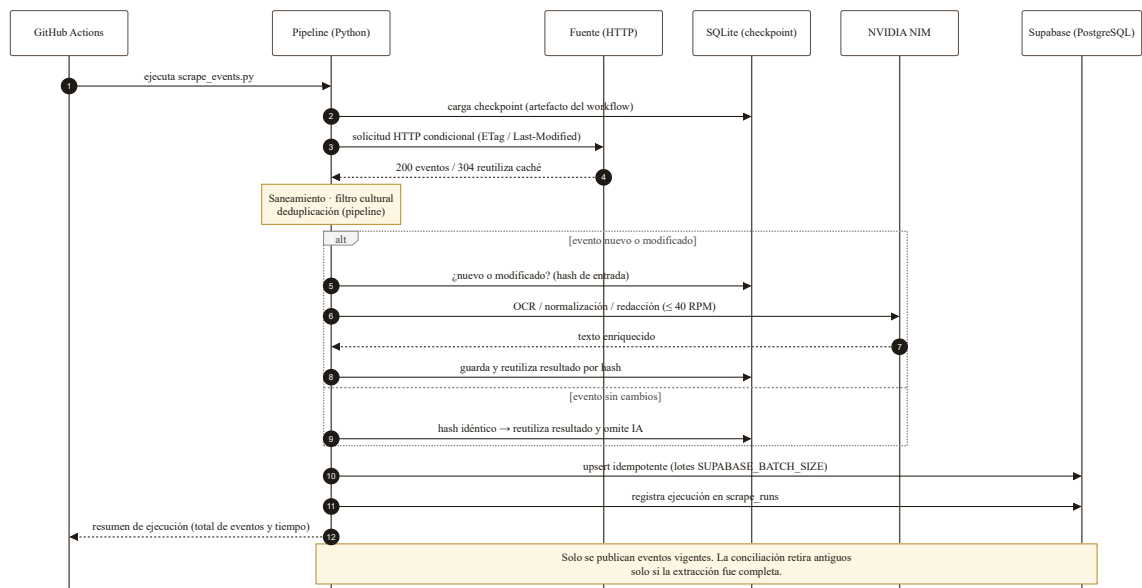


Figura 5: Secuencia: extracción, enriquecimiento y publicación (editable: Diagramas/04_secuencia.mmd, Mermaid).

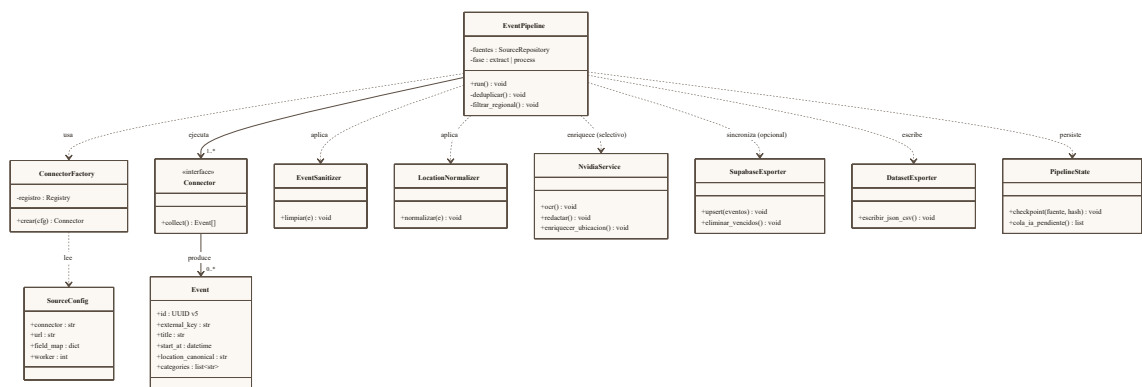


Figura 6: Diagrama de clases conceptual del núcleo del scraper (editable: Diagramas/05_clases.mmd, Mermaid).

6.4. Pendientes de UML

Pendiente de completar por el equipo: si el docente exige una versión gráfica adicional con notación UML estricta, regenerarla a partir de los archivos editables entregados y sustituir las figuras correspondientes.

7. Requisitos no funcionales

7.1. Requisitos definidos en Fase 1 (MoSCoW)

7.2. Estado real por requisito

ID	Estado	Evidencia verificable
RNF-01	Parcial	Navegación adaptativa implementada; sin evaluación de usabilidad registrada. Pendiente.

ID	Estado	Evidencia verificable
RNF-02	Cumplido	Ionic/Angular responsive; navegación dual móvil/escritorio (<code>MainLayoutComponent</code>), grid con breakpoints.
RNF-03	Parcial	Caché condicional, bulkhead por dominio y checkpoints optimizan la extracción; falta medición de tiempos del frontend con datos reales. Pendiente.
RNF-04	Parcial	RLS en todas las tablas y secretos solo en entorno/GitHub Secrets; las pantallas de login y registro aún son simuladas.
RNF-05	Parcial	El modelo prevé roles (<code>profiles.role</code>) y reglas RLS; el control de acceso funcional en la aplicación está pendiente.
RNF-06	Parcial	GitHub Actions con ejecución manual y cron diario activo desde el 24-09-2026 (verificado: corridas #9–#11 del 25 al 27-09). Pendiente: ejecución automática completa en verde tras corregir la prueba dependiente del tiempo (Plan de pruebas §5.1).
RNF-07	Cumplido (diseño)	Paginación por página en conectores, upsert idempotente y SQLite como buffer.
RNF-08	Cumplido (pipeline)	Sanitización, filtro cultural, normalización de ubicación, deduplicación y UUID deterministas.
RNF-09	Cumplido	Documentación interna (<code>docs/</code>), patrones documentados y 14 módulos de pruebas.
RNF-10	Cumplido	Contenido en español; división administrativa chilena (regiones/comunas).
RNF-11	Parcial	Portabilidad adoptada por el equipo: GitHub Actions (scraper), Vercel (web) y Capacitor 8 + Android Studio (Android); sin Docker por decisión del equipo (ver Sección 9).
RNF-12	Parcial	Accesibilidad como criterio de filtro en el frontend; auditoría formal pendiente. Pendiente.

Cuadro 6: Estado real de cada requisito no funcional.

7.3. Requisitos declarados y no implementados

Para evitar inconsistencias en la evaluación se declara explícitamente: no existe autenticación real en el frontend (pantallas simuladas); las notificaciones (RF-13), el panel de administración (RF-14) y la moderación (RF-11/RF-12 del catálogo funcional) no tienen implementación en el código entregado; las estadísticas (RF-17) quedaron *Won't have* en Fase 1; Docker no está implementado y el equipo decidió no utilizarlo (Sección 9).

7.4. Limitaciones técnicas del desarrollo

El desarrollo se realiza en un contexto académico sin presupuesto, lo que impone dos limitaciones operativas visibles en el diseño del sistema:

- **Cuota de la capa gratuita de IA (principalmente tiempo):** el enriquecimiento con IA se apoya en **NVIDIA Build**, plataforma de prototipado con modelos en la nube de uso gratuito pero sujeto a una cuota global de **40 peticiones por minuto (RPM)**. Esta restricción de tiempo condiciona el volumen de eventos que pueden procesarse por corrida y explica dos

ID	Categoría	Requerimiento	Prioridad
RNF-01	Usabilidad	Interfaz clara para ciudadanos y organizadores	Must
RNF-02	Compatibilidad	Diseño responsive / multiplataforma	Must
RNF-03	Rendimiento	Tiempos de carga aceptables con alto volumen	Should
RNF-04	Seguridad	Protección de datos en tránsito y reposo	Must
RNF-05	Seguridad	Control de acceso por rol	Must
RNF-06	Disponibilidad	Disponibilidad estable según hosting definido	Should
RNF-07	Escalabilidad	Crecimiento sostenido de datos	Should
RNF-08	Calidad de datos	Normalización y consistencia entre fuentes	Must
RNF-09	Mantenibilidad	Código mantenible y documentado	Should
RNF-10	Localización	Contenido en español y contexto chileno	Must
RNF-11	Portabilidad	Despliegue reproducible (GitHub Actions)	Could
RNF-12	Accesibilidad	Buenas prácticas de accesibilidad web	Could

Cuadro 5: Requisitos no funcionales priorizados (matriz MoSCoW de Fase 1).

decisiones de diseño ya documentadas: la IA se aplica *selectivamente* (solo a eventos nuevos o modificados, con control de costo y latencia) y el pipeline es *reanudable* (checkpoints SQLite por fuente y ficha, que evitan repetir trabajo tras un corte o una interrupción por cuota).

- **Sin infraestructura propia de cómputo (hardware):** el equipo no dispone de un servidor donde alojar un modelo de IA especializado (OCR de afiches y análisis de texto); por eso se utiliza NVIDIA Build como alternativa gratuita de prototipado en la nube en lugar de un despliegue local o dedicado.

Otras limitaciones visibles del estado actual —frontend conectado a datos de ejemplo (*mock*), autenticación simulada y despliegue web aún no efectuado— no son restricciones de infraestructura sino trabajo pendiente ya declarado, trazado en las secciones de estado real y de pruebas. La ejecución programada del scraper está activa (cron diario desde el 24-09-2026); su robustez depende de corregir la prueba dependiente del tiempo documentada en el plan de pruebas.

Pendiente de completar por el equipo. decisión futura sobre migrar el enriquecimiento con IA a infraestructura propia o a una cuota pagada; en tal caso, las restricciones de esta subsección y las decisiones de diseño asociadas se reevaluarán.

8. Pruebas

8.1. Pruebas existentes del scraper

Comando oficial (README del scraper): `python -m unittest discover -s tests -v`. La carpeta `tests/` contiene 14 módulos (aproximadamente 1.900 líneas) con carácter unitario y de módulo: usan dobles de prueba del cliente HTTP, del repositorio de fuentes y del exportador, según la inyección de dependencias documentada.

8.2. Pruebas existentes del frontend

El frontend configura Vitest con jsdom (comando `ng test`) e incluye pruebas de humo de creación de componentes y servicios. No cubre todavía la lógica de filtros ni de servicios.

Módulo	Qué valida
<code>test_core.py</code>	Configuración, mapeo WordPress, deduplicación/filtro y exportación
<code>test_event_sources.py</code>	Conectores (Fisa, EventOn, Generic, Ticketplus, HtmlCards) y catálogo mínimo de 10 fuentes activas
<code>test_supabase.py</code>	Transformación idempotente al esquema de Supabase
<code>test_resumable_pipeline.py</code>	Checkpoints, reanudación y pausas NVIDIA
<code>test_performance.py</code>	Caché condicional, lecturas Maipú, archivos operativos y pipeline paralelo
<code>test_locations.py</code>	Normalización de ubicación (reglas y NVIDIA)
<code>test_content_enrichment.py</code>	Enriquecimiento editorial
<code>test_nvidia.py</code>	Respuestas y límites NVIDIA
<code>test_sanitization.py</code>	Limpieza HTML y vigencia
<code>test_cultural_filter.py</code>	Filtro de pertinencia cultural
<code>test_jsonld.py</code>	Extracción JSON-LD
<code>test_availability.py</code>	Disponibilidad de eventos
<code>test_analytics.py</code>	Métricas operativas
<code>test_source_validation.py</code>	Reglas del validador de fuentes

Cuadro 7: Pruebas unitarias y de módulo existentes en el scraper.

8.3. Pruebas pendientes

8.4. Registro de resultados

9. Docker: estado actual y plan

9.1. Estado real verificado

Conclusión: Docker no está implementado en ninguna parte del sistema (se declara explícitamente porque el anexo metodológico exige documentar su estado). La estrategia de despliegue adoptada por el equipo (decisión comunicada el 24-09-2026) no contempla contenedores:

- **Scraper:** ejecución local en Windows (`instalar.bat`, `ejecutar.bat`) y automatización en **GitHub Actions**; el workflow instala por sí mismo Python, Playwright y Chromium.
- **Web:** despliegue en **Vercel** (hosting previsto en el design brief de Fase 1 y confirmado por el equipo); build de producción con `ng build`. A la fecha no existe evidencia de un despliegue ejecutado.
- **Android:** compilación de la aplicación con **Capacitor 8** y **Android Studio** (dependencias de Capacitor presentes en `package.json`).
- **Base de datos:** servicio administrado de Supabase, sin contenedores.

9.2. Plan de implementación (alternativa documentada, no adoptada)

El equipo decidió no utilizar Docker en la estructura de despliegue actual. El siguiente plan se conserva documentado como alternativa futura y como respuesta al punto 5 del anexo metodológico; su eventual implementación queda **pendiente de completar por el equipo** solo si la estrategia de despliegue cambiara.

1. **Contenedor del scraper:** imagen base `python:3.12-slim`; instalación de `requirements.txt` y Chromium de Playwright dentro de la imagen; volúmenes para `state/` (checkpoints) y `data/`

#	Prueba	Alcance	Tipo
1	Unitarias de <code>EventosService</code> (filtros y orden)	Frontend	Unitaria
2	Unitarias de recomendaciones y preferencias	Frontend	Unitaria
3	Integración frontend-Supabase (vista pública)	Frontend + BD	Integración
4	Integración scraper-Supabase (upsert idempotente)	Scraper + BD	Integración
5	Rendimiento: carga del listado con volumen real	Frontend	Rendimiento
6	Rendimiento: pipeline completo (usar <code>run_summary.json</code>)	Scraper	Rendimiento
7	Seguridad: verificación de políticas RLS por rol	BD	Seguridad
8	Seguridad: saneamiento de entradas de usuario	Scraper/Frontend	Seguridad
9	Accesibilidad: auditoría Lighthouse/axe	Frontend	No funcional
10	Registro de resultados y trazabilidad	Proceso	Proceso

Cuadro 8: Plan de pruebas pendiente. Responsables, fechas y criterios de aceptación: **pendiente de completar por el equipo.**

Fecha	Suite ejecutada	Resultado	Responsable
—	—	Pendiente de completar por el equipo	—

Cuadro 9: Registro de ejecución de pruebas (sin resultados inventados).

(respaldos); configuración por variables de entorno según `.env.example`, sin credenciales embebidas; ejecución por fases (`--phase extract` / `--phase process`).

2. **Contenedor del frontend:** build multi-etapa (etapa Node con `ng build`; etapa de servicio estático con `nginx`), con variables de entorno de runtime para el endpoint del backend futuro.
3. **Composición opcional:** `docker-compose.yml` de desarrollo para el frontend servido y el scraper como job; la base de datos permanece como servicio administrado de Supabase.

Criterios de aceptación propuestos: (1) `docker build` de ambas imágenes sin credenciales; (2) una ejecución completa del scraper en contenedor produce `data/events.json` y registra la corrida en `scrape_runs`; (3) el frontend en contenedor responde en el puerto definido; (4) README actualizado con las instrucciones `docker compose up`.

10. Innovación

10.1. ¿Qué problema resuelve?

En Chile la oferta de eventos culturales está fragmentada en decenas de publicadores independientes (municipalidades, centros culturales, ticketeras, universidades y portales regionales), cada uno con formatos, fechas y estructuras distintos: agendas WordPress, APIs, carteleras HTML e imágenes de afiches. El ciudadano debe revisar sitio por sitio y muchos eventos —especialmente municipales y gratuitos— no llegan a las plataformas comerciales. La magnitud del problema

Componente	Dockerfile	docker-compose	Contenedores en ejecución
Scraper (Python)	No	No	No
Frontend (Angular/Ionic)	No	No	No
Base de datos	— (servicio administrado Supabase)	No	No

Cuadro 10: Estado de Docker verificado por inspección de ambos entregables: no implementado.

quedó documentada en el propio proyecto: el inventario de fuentes levantó 390 candidatas, de las cuales se integraron 20 permanentes más 1 estacional mediante conectores específicos.

10.2. ¿Qué hace diferente a la solución?

- **Cobertura municipal y regional:** prioriza agendas culturales públicas (municipios, centros culturales, universidades), no solo ticketeras comerciales (tabla de 21 fuentes del README del scraper).
- **Enriquecimiento con IA selectivo:** OCR de afiches, normalización de ubicaciones y redacción editorial con NVIDIA NIM, aplicado solo a eventos nuevos o modificados, con control de costo y latencia (40 RPM globales).
- **Calidad de datos como regla, no como tarea:** sanitización, filtro de pertinencia cultural, normalización geográfica y deduplicación obligatorias para toda fuente; un conector nuevo no puede saltárselos.
- **Pipeline reanudable y auditable:** checkpoints SQLite por fuente y ficha; reanudación tras cortes y bitácora en `scrape_runs`.
- **Publicación siempre vigente:** reutiliza eventos sin cambios y retira registros antiguos solo cuando la extracción fue completa y segura.
- **Recomendación con señales de interés:** vista “para ti” en lugar de un listado estático, y experiencia multiplataforma desde el mismo código.

10.3. ¿Qué valor agrega?

Para el ciudadano, un solo lugar para descubrir eventos con búsqueda, filtros (región, comuna, fecha, gratuito, modalidad, accesibilidad), mapa y calendario. Para los organizadores públicos, visibilidad sin costo para agendas municipales y de centros culturales. Para el ecosistema de datos, un catálogo normalizado con procedencia verificable (`event_provenance` conserva texto original y OCR). Para lo académico, la aplicación real de IA generativa en un pipeline de datos con control de costos y de patrones de arquitectura documentados y probados.

Declaración de honestidad: los resultados de operación reales (número de eventos publicados, tiempos, usuarios) no se afirman en este documento y deben registrarse con datos de ejecución cuando el equipo los genere.

10.4. Modelo de negocio (propuesto, no implementado)

El equipo definió dos líneas de sostenibilidad, que a la fecha son **propuestas de diseño** y no funcionalidades implementadas ni comprometidas contractualmente:

1. **Publicación no invasiva con redireccionamiento de información (línea principal):** el catálogo se alimenta exclusivamente de fuentes públicas y se redirige al usuario hacia la fuente original de cada evento (municipalidad, centro cultural o ticketera) para consultarlo o reservarlo. La plataforma no intermedia pagos ni cobra por la publicación: el valor para el

ciudadano es el descubrimiento unificado y el valor para el organizador público es la visibilidad sin costo, lo que se alinea con la procedencia verificable de `event_provenance`.

2. **Contacto directo con organizadores por visibilidad mejorada (línea a evaluar):** como opción futura menos probable, los organizadores podrían pagar por destacar sus eventos en la plataforma (mayor visibilidad en búsquedas y portada). Esta línea queda **por decidir por el equipo**: exige validar aspectos legales, de precio y de imparcialidad del catálogo antes de adoptarse.

Ninguna de las dos líneas compromete el carácter público y gratuito del descubrimiento de eventos, que es el objetivo central del proyecto.

11. Trazabilidad documental

Cada afirmación de este documento proviene de las siguientes fuentes verificables:

Sección	Evidencia utilizada
Arquitectura general, scraper y frontend	README y docs/ARCHITECTURE.md, docs/RESUMABLE_PIPELINE.md, docs/SUPABASE.md del scraper; package.json, app.routes.ts, README y documentación LaTeX del frontend; diagramas de Fase 2.
Modelo de datos	docs/DATABASE.md, docs/SUPABASE.md, docs/evento_cultural_modelo.dbml, diccionario de base de datos (DOCX) y DB_evento_cultural.pdf (Fase 1).
Diagramas UML	Rutas y servicios reales del frontend; clases y patrones documentados del scraper; matriz MoSCoW de Fase 1. Los seis diagramas (01–06) comparten estándar visual y fueron pulidos y reexportados el 27-09-2026 (§16 del registro de cambios).
Requisitos no funcionales	Matriz Requerimientos_Funcionales_NoFuncionales_MoSCoW.xlsx (Fase 1) y evidencia de código por requisito.
Pruebas	Carpeta tests/ del scraper (14 módulos, aproximadamente 1.900 líneas), sección “Pruebas” de su README y configuración Vitest del frontend.
Docker	Inspección completa de ambos entregables (sin Dockerfile ni compose) y CONTENIDO_DEL_REPOSITORIO.md.
Innovación	Design brief de Fase 1, matriz MoSCoW, tabla de fuentes del README del scraper y funcionalidad “para ti” del frontend.

Cuadro 11: Trazabilidad entre secciones del documento y evidencia del repositorio.

El detalle de altas, cambios y pendientes de toda la documentación se registra en `REGISTRO_CAMBIOS_DOCUMENTOS` en la raíz de `CAPSTONE_`.